

Phase 1 · Step 1.3 — 비용함수(Cost Function)

Routing3D · turn penalty / 클리어런스(가산 페널티) / 단 분리 · 모듈 cost.py

1. 개요

직교 A* 의 '이동 비용'을 정의한다. 기본 이동비용(셀 = cell_mm) 위에 다음을 더한다.

- Turn penalty: 진행 방향이 바뀔 때 가산 → 직각 회전 최소화(엘보 줄이기).
- 클리어런스 페널티: 장애물에 가까운 셀일수록 가산 → 벽에서 떨어져 지나가게 유도.
- 단(段) 분리: z 레벨별 가산 → 특정 단(배관 랙)으로 유도/회피.

2. 핵심 설계 결정 — 왜 '보너스'가 아니라 '페널티'인가

계획서 초안은 클리어런스를 '여유 셀 수에 비례한 비용 감산(보너스)'으로 표현했다. 그러나 비용을 감산하면 한 칸 이동 비용이 cell_mm 보다 작아질 수 있어, 맨해튼×cell_mm 휴리스틱이 실제 비용을 과대평가하게 되고 → A* 의 admissibility(최적성)가 깨진다.

그래서 동일한 목적(벽에서 멀어지기)을 '장애물 근접 시 가산하는 페널티'로 구현한다. 모든 가산항이 ≥ 0 이므로 이동 비용 $\geq \text{cell_mm}$ 이 보장되고, 맨해튼 휴리스틱이 admissible & consistent 하게 유지된다(A* 최적성 보존).

3. 전체 흐름도

```
RouteParams(가중치) + OccupancyMap
|
▼ CostModel(occ, params)          # 생성 시 클리어런스 맵 1 회 사전계산
|
▼ move_cost(to_cell, prev_off, move_off)
= cell_mm
+ (w_turn          if 방향 바뀜)
+ 클리어런스 페널티(to_cell)    # 장애물에 가까울수록 큼
+ 단 분리 페널티(to_cell.z)
```

4. 핵심 알고리즘

4.1 clearance_map — bounded distance transform

각 셀에서 '가장 가까운 장애물까지의 거리(셀)'를 max_radius 로 상한해 계산한다. 장애물을 1 단계씩 팽창하며 새로 덮이는 셀에 그 단계 번호를 거리로 기록한다.

```
장애물 셀 = 거리 0, 멀리 떨어진(끝까지 안 덮인) 셀 = max_radius
current = grid(장애물)
for d in 1..max_radius:
    dilated = _dilate_once(current, offsets)
    newly = dilated & ~current      # 이번 단계에 처음 덮인 셀
    dist[newly] = d
    current = dilated
```

4.2 cell_penalty — 목적지 셀 가산 페널티

```
pen = 0
if clearance 있음 and d < clearance_radius:
    pen += w_clear × (clearance_radius - d)  # 가까울수록(d 작을수록) 큼, d=0 최대
if w_tier:
    pen += w_tier.get(cell.z, 0)            # 단 분리
```

4.3 move_cost — 이동 1 회 총 비용

```
c = cell_mm
if prev_off is not None and move_off != prev_off:
    c += w_turn                                # 방향 바뀜 = 회전
c += cell_penalty(to_cell)
return c
```

4.4 astar_weighted 연동 — 방향을 가진 상태

turn penalty 는 '직전 진행 방향과 다른 방향으로 꺾을 때' 부과되므로, 같은 셀이라도 어느 방향으로 들어왔는지에 따라 이후 비용이 달라진다. 따라서 탐색 상태를 (셀, 진입방향)으로 확장한다. 진입방향은 NEIGHBORS_6 인덱스(0~5), 시작 상태는 -1(방향 없음). g / closed / came_from 모두 이 상태 단위로 관리한다(astar.py 의 astar_weighted).

5. 자료구조 / 클래스

이름	종류	설명
RouteParams	dataclass	비용 가중치 묶음(아래 표). 음수면 ValueError.
CostModel	class	occ+params 로

		이동비용/휴리스틱 계산. 클리어런스 맵 사전계산.
clearance	np.int16[]	CostModel 속성: 셀별 장애물까지 거리(없으면 None).

6. 주요 함수

함수 / 메서드	역할
clearance_map(occ, max_radius, connectivity=6)	셀별 장애물 근접 거리맵(상한 max_radius).
CostModel(occ, params)	생성 시 클리어런스 맵 1 회 사전계산.
CostModel.cell_penalty(cell)	클리어런스 근접 + 단 분리 가산 패널티.
CostModel.move_cost(to, prev_off, move_off)	이동 1 회 총 비용(기본+회전+페널티).
CostModel.heuristic(cell, goal)	맨해튼×cell_mm. admissible & consistent.

7. 주요 변수 / 파라미터 (RouteParams)

필드	기본값	설명
cell_mm	50.0	셀 1 칸 이동 기본 비용(mm).
w_turn	500.0	회전 1 회당 가산(mm, =셀 10 칸). 클수록 회전 강하게 회피.
w_clear	10.0	클리어런스 패널티 계수(mm/셀). 0=비활성.
clearance_radius	2	페널티 적용 최대 근접 거리(셀). 이보다 멀면 0.
clearance_connectivity	6	거리 측정 이웃(6 맨해튼 / 26 체비셰프).
w_tier	{}	단 분리 가중치 {z 셀인덱스: 가산 mm}.

8. 실행 명령어

```
# 비용함수 적용 A* (회전·클리어런스 패널티)
.\.venv\Scripts\python.exe -m routing3d_py.astar ^
  --region 195000 8000 14000 205000 12000 16000 --cell-mm 50 ^
  --start 195300 8300 14775 --goal 204700 11700 14775 ^
  --w-turn 500 --w-clear 10 --clearance 2 ^
  --screenshot python_experiments/out/route_cost.png

# 단위 테스트
.\.venv\Scripts\python.exe -m pytest python_experiments/tests/test_cost.py -v
```

```
# 코드에서 사용
from routing3d_py import astar_weighted, RouteParams
res = astar_weighted(occ, start, goal,
                    RouteParams(w_turn=500, w_clear=10, clearance_radius=2))
```